

AI651 – Deep Learning for Space, Time and Graphs

Assignment 02

Student Name: Maria Rafique
Student ID: 25280077
Semester: Fall 2026
Github Code Link: PA2 Motion or Memory

Syed babar ali school of science and engineering, LUMS

October 30, 2026

Contents

1 Question 1:- River Forecasting Proposal **4**
1.1 Architecture Diagram . . . 4
1.2 Assumptions . . . 6
1.2.1 Provided facts (not assumptions) . . . 6
1.2.2 Data conditions . . . 6
1.2.3 River dynamics . . . 8
1.2.4 Operations and training . . . 9
1.3 Forecasting task and data specification . . . 10
1.3.1 Task . . . 10
1.3.2 How far back the model needs to look . . . 10
1.3.3 Inputs and outputs . . . 11
1.3.4 Putting all sources on one timeline . . . 12
1.3.5 What the model is allowed to know at issue time . . . 13
1.4 Model computations and design justification . . . 13
1.4.1 Notation . . . 13
1.4.2 Block 1: per-source input projection . . . 14
1.4.3 Block 2: temporal encoder f_{time} (four dilated causal T blocks) . . . 14
1.4.4 Block 3: directed message passing f_{graph} (repeated $k = 1, \ldots, K$, $K = 4$) . . . 14
1.4.5 Block 4: direct forecast head g . . . 15
1.4.6 Training objective . . . 16
1.4.7 How each source reaches each forecast . . . 16
1.4.8 Physical assumptions versus learned mappings . . . 17
1.5 Operation and exceptional conditions . . . 17
1.5.1 (i) Missing observations . . . 17
1.5.2 (ii) Delayed observations . . . 18
1.5.3 (iii) Announced future releases . . . 18
1.5.4 (iv) Differences between scheduled and actual releases . . . 18
1.5.5 Running every ten minutes is feasible . . . 19
1.6 Uncertainty, limitations and additional information . . . 19
1.6.1 Uncertainties and limitations . . . 19
1.6.2 Additional information requested . . . 20

2 Question 2: Motion or Memory? **21**
2.1 Setup and checks . . . 21
2.2 Q2.1 . . . 22
2.3 Q2.2 . . . 24

2.4 Q2.3 . . . 25
2.5 Q2.4 . . . 26
2.6 Q2.5 . . . 26
2.7 Q2.6 . . . 27
2.8 Q2.7 . . . 29
2.9 Q2.8 . . . 29
2.10 Q2.9 . . . 31
2.11 Q2.10 . . . 31
2.12 Q2.11 . . . 32
2.13 Q2.12 . . . 32
2.14 Q2.13 . . . 33
2.15 Q2.14 . . . 33
2.16 The final model . . . 34
2.17 Q2.15 . . . 35
2.18 Q2.16 . . . 37
2.19 Q2.17 . . . 37

3 Use of AI **38**

1 Question 1:- River Forecasting Proposal

Every 10 minutes the system will issue a probabilistic forecast of the water level at sensors S1, S2, S3, S4 for up to one hour ahead. It will read the last 3 hours of water levels reading, rainfall and dam releases and the dam schedule future announced. Its spatio-temporal problem with irregular placement of sensors so I am combining temporal convolutional networks (TCNs) with directed graph message passing(idea i got from tutorial).

I have put the last three hours of levels, rainfall and dam releases, plus the dam schedule already announced, on one aligned grid. Four dilated causal temporal blocks can summarize each location's history, four directed graph layers will pass those summaries downstream, and a direct head predicts the change from the last observed level at all six lead times at once.

Throughout the report i will use $\Delta = 10$ min, a lookback of $L = 18$ columns (3 h), a horizon of $H = 6$ columns (1 h), and a grid of $T = L + H = 24$ columns. The grid has $N = 6$ nodes (Dam, S1, T1, S2, S3, S4) and $C = 9$ channels per cell, and the hidden width is 64.

Table 1: System design choices, values, and justifications

Choice	**Value**	**Reason**
Issuance	Every 10 min, at time t_0	continuous real-time monitoring.
Horizon (H)	6 steps (t_0+10 to t_0+60 min)	t_0+60 is required; intermediate steps adding trend information and training signal.
Lookback (L)	18 steps (3 h)	Covering the slowest dam-to-S_4 travel plus the rain-to-river lag.
Graph	6 nodes, 5 directed edges, $K = 4$ layers	Longest path Dam $\rightarrow S_4$ is 4 hops.
Forecast mode	Direct multi-output, not autoregressive	so it doesnot compound error; sp no future rain or levels needed
Output	Mean and standard deviation per station and lead	Flood preparedness needs a range; S_3 and S_4 are less certain

1.1 Architecture Diagram

Figure 1 showing the full system. Follow it from top to bottom. The four data sources passing through the alignment stage which apply the t_0 cut-off and quality control, bins each source onto the ten-minute grid, maps the rain gauges onto river stations, and concatenate everything into one tensor $X \in \mathbb{R}^{B\times T\times N\times C}$. Block 1 is projecting the dam row and the station rows separately into a shared 64-dimensional space. Block 2 is encoding each node's history over time. Block 3 passes information down the river graph, and Block 4 producing the forecast. The last valid level $\tilde{y}$ bypassing the model and is added back in Block 4, so that the network only predicts the change from now. The training objective is used only in training.

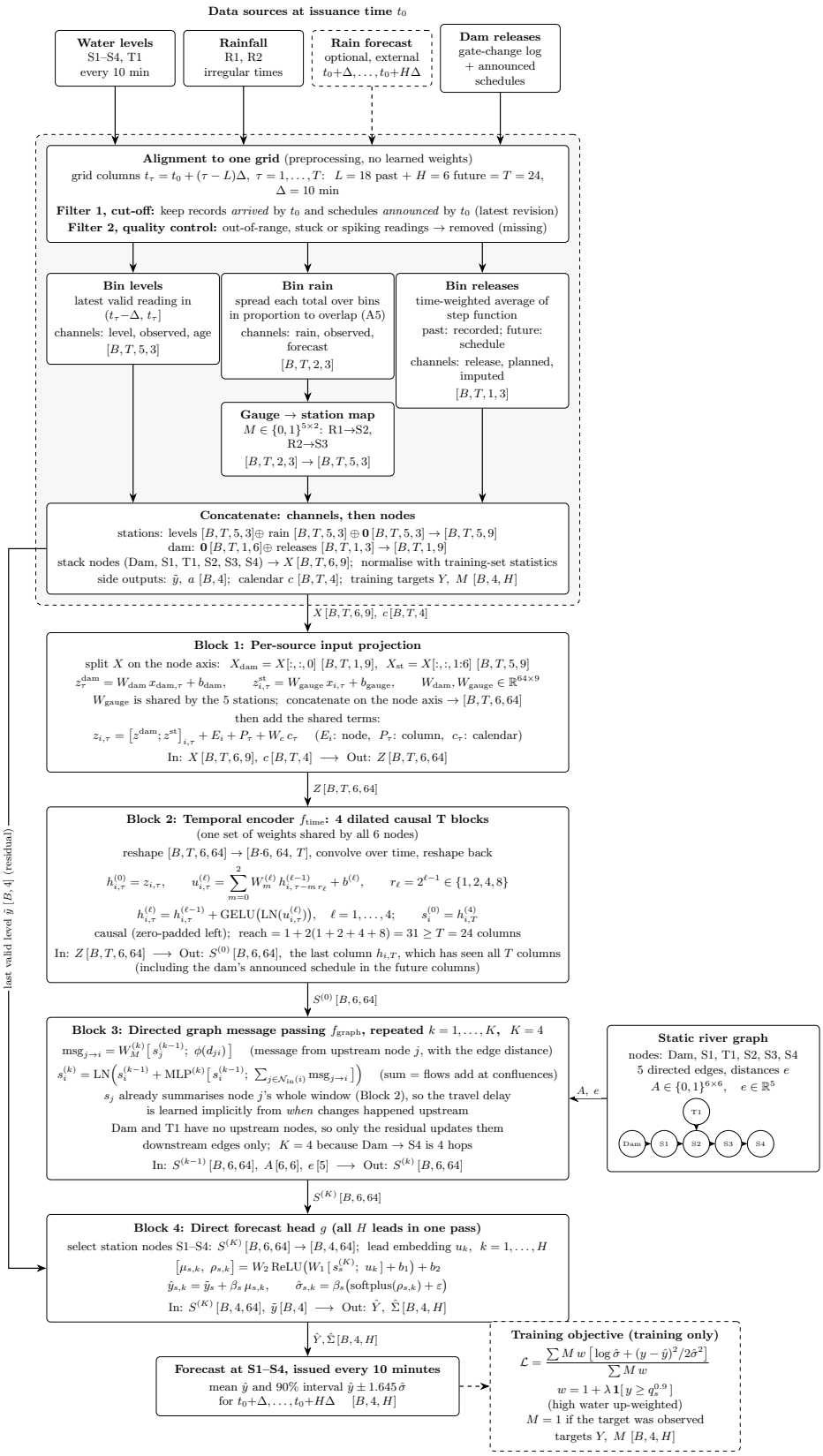


Figure 1: Forecasting architecture with tensor shapes, alignment, projection and reshaping steps.

- **Data Sources & Tensor Alignment**: Incoming streams from stations (S_1–S_4, T_1),

rain gauges (R_1, R_2), and reservoir control logs have been resampled into a uniform 10-minute grid. I have projected into node-level feature tensors $\mathbf{X} \in \mathbb{R}^{B \times N \times T_{hist} \times F}$, where $N = 6$ nodes (S_1, S_2, S_3, S_4, T_1, and the Reservoir/Dam), T_{hist} is the historical lookback window, and F is the feature dimensionality.

- **Spatial Message Passing**: Graph convolutional layers mixes upstream hydrological influences across directed river connectivity edges defined in adjacency matrix $\mathbf{A}$.
- **Temporal Convolutions**: Dilated causal 1D convolutions process time series independently per node without information leakage from the future and expanding the receptive field to capture multiple hour flood wave travel times.
- **Forecast Mapping**: Output tensors are then fed into a shared linear regression head to generate 6 sequential direct 10 minute steps (+10m to +60m) for stations S_1 through S_4.

1.2 Assumptions

1.2.1 Provided facts (not assumptions)

ID	Fact from the handout
F1	Historical water levels at S1–S4 and at tributary station T1, recorded every ten minutes.
F2	Historical rainfall at R1 and R2, with irregular timestamps and its gauge locations.
F3	Recorded reservoir releases, and future release schedules announced before each forecast, with announcement and effective times.
F4	Station locations, river connectivity and distances along the river.
F5	The reservoir feeding the river above S1; a monitored tributary (T1) joining between S1 and S2; an unmonitored tributary joins between S2 and S3. Stations are unevenly spaced and travel times change with river conditions.
F6	Precise travel times and the unmonitored tributary's inflow are not supplied.
F7	Future rainfall and future water levels are not available at issuance. Extra information can be requested but again it is not guaranteed.

Everything else is assumption. Each one below is what I assumed and why it is reasonable, and what it changes in the design.

1.2.2 Data conditions

The following assumptions describe the data I expect to receive from the authority. Since the assignment does not specify every detail about data quality, reporting frequency or measurement conditions, I make these assumptions to define how my forecasting system will work.

A1 **Record length.** I assume that at least three years of historical data are available, including several high-water events. This provides examples of both normal river

conditions and unusually high water levels, which are important for training the model to recognise different river behaviours. I split the data chronologically by year into training, validation and test sets, and give greater importance to high-water periods in the training loss. If less data is available, I can reduce the model's hidden width and report that its ability to predict extreme flood events has not been fully tested (U3).

A2 **Common time reference.** I assume that all data sources use the same synchronised clock (UTC). Each water-level timestamp represents the end of its ten-minute measurement interval. Therefore, grid column τ represents the interval $(t_\tau - \Delta,\, t_\tau]$. Using a common timeline allows me to combine water levels, rainfall and reservoir releases correctly. I align all observations to the same ten-minute grid before passing them to the model, using $\Delta = 10$ minutes.

A3 **Reporting delays.** I assume that most water-level readings arrive within ten minutes, while rainfall reports may arrive up to 30 minutes late. Some readings may also be delayed indefinitely or never arrive. Since communication through radio and cellular networks can be unreliable, particularly during severe weather, I only use readings that have arrived by the forecast issue time t_0. Each input includes an observation flag to indicate whether a reading is available and an age value showing how old the latest reading is. During training, I simulate similar delays so that the model can handle them during operation (Section 5).

A4 **Measurement quality.** I assume that sensor readings can contain sudden spikes, stuck values, missing intervals or saturation during extreme water levels. To avoid treating faulty measurements as reliable observations, I apply basic quality checks before using the data. Readings that fail these checks are treated as missing rather than being used directly. I do not attempt to guess or replace faulty measurements with corrected values.

A5 **Meaning of rainfall readings.** I assume that each rainfall report gives the total rainfall depth (in mm) accumulated since the previous report. Since reports may arrive at irregular intervals, I assume that rainfall is distributed evenly across the reporting interval. This provides a consistent way to convert rainfall reports into ten-minute inputs. I distribute each reported amount across the grid intervals according to how much of each interval it overlaps. If a rainfall report has not yet covered a particular interval, I mark that interval as unknown rather than treating it as a dry period. If the gauges instead record individual tipping-bucket events, only this rainfall preprocessing step needs to change.

A6 **Water-level reference.** I assume that each station measures water level in metres relative to its own local reference point (datum). Since different stations may use different reference levels, their raw measurements may not be directly comparable. I therefore normalise water levels separately for each station using its training-set statistics. The model predicts the change from the latest valid water level rather than predicting the absolute level from scratch.

A7 **Reservoir releases and schedules.** I assume that recorded releases represent the actual reservoir outflow in m^3/s and remain constant between gate changes. A release schedule describes the planned outflow from its stated effective time onward.

If a new schedule is announced, it replaces the earlier schedule from the relevant effective time. I also assume that schedules normally cover at least the next hour. I align historical releases to the ten-minute grid using their time-weighted average. For future time steps, I use the latest schedule available at forecast issue time t_0. If a future interval is not covered by a schedule, I carry forward the last known release as a fallback and mark it as imputed. This ensures that planned releases are used as future inputs without being mistaken for actual future measurements.

A8 River distances. I assume that the supplied distances represent the lengths measured along the river channel between connected nodes, including the distance from T1 to its confluence with the main river. Since these distances describe the actual river paths rather than straight-line distances between station coordinates, I use them as edge features in the directed graph. This helps the model distinguish between nearby and more distant upstream locations.

1.2.3 River dynamics

A9 Downstream influence only. I assume that water and its effects move downstream, while backwater effects from lower stations are negligible. Since the stations are separated by river reaches with a downhill slope, I represent the river as a directed graph in which information flows only from upstream to downstream. The graph edges are therefore Dam→S1, S1→S2, T1→S2, S2→S3 and S3→S4.

A10 Conservation and rating curves. I assume that flow (m^3/s) is conserved, meaning that flows from different upstream sources combine at confluences after accounting for their travel delays. However, water levels do not simply add because each station has its own rating curve: a monotonic, nonlinear and relatively stable relationship between flow and water level. Based on this, I sum incoming graph messages at confluences and allow the model to learn the relationship between flow-related information and water level rather than imposing a fixed conversion.

A11 Travel time. I assume that travel time between two points depends on the distance along the river and the speed of the travelling flood wave. Wave speed increases with flow, and I use a minimum wave speed of approximately $c_{\min} = 1.2$ m/s as a working assumption. Based on this value, I assume that the distance from the Dam to S4 is at most approximately 10.8 km, while the distance from S2 to S4 is at most approximately 6.5 km. Since the assignment provides river distances but does not specify travel times, I use these estimates to justify the historical lookback of $L = 18$ time steps (Section 3). If the supplied distances are longer, I increase the lookback accordingly. The model learns the actual predictive delay from historical data rather than relying on a fixed travel-time value.

A12 Rain-to-river lag. I assume that rainfall takes up to approximately one hour to reach the river as runoff. This is based on the assumption that the surrounding catchments are relatively small and steep. As a result, the historical rainfall inputs within the lookback window can help the model capture recent rainfall effects. Rainfall occurring after t_0 is expected to have a limited influence within the one-hour forecasting horizon.

A13 Where R1 drains. I assume that R1 measures rainfall over the land between the dam and S2, with the resulting runoff entering the main river at S2. Rainfall falling directly into the reservoir is treated differently because it is stored and reaches the river through reservoir releases. Based on this assumed drainage arrangement, I assign R1 rainfall to S2 in the drainage mapping, while rainfall over the reservoir influences the model through the dam node and its release information.

A14 Where R2 drains. I assume that R2 measures rainfall over the catchment of the unmonitored tributary, which joins the main river at S3. I also assume that the tributary flow consists of a relatively slow baseflow component and a rainfall-driven component that is correlated with R2. Since the tributary has no monitoring station, its actual flow is not directly available. I therefore assign R2 rainfall to S3 in the drainage mapping and allow the model to learn its relationship with downstream water levels. Any contribution that cannot be explained by R2 and the available upstream information remains a source of uncertainty at S3 and S4 (Section 6). S1, T1 and S4 do not receive separate rainfall channels. The model uses node embeddings to distinguish between a location with no assigned rain gauge and a location where an expected reading is missing.

A15 Stable system. I assume that the river channel shape, rating curves and dam operating rules remain reasonably stable throughout the historical record. I also assume that there are no major water withdrawals or additional unaccounted inflows. Slower seasonal effects, such as snowmelt, are represented indirectly through recent water-level history and calendar features. This allows me to train one model using the full historical record and retrain it periodically as new data become available.

1.2.4 Operations and training

A16 Infrastructure. I assume that all incoming data are available through a central database and that a standard CPU server runs the forecasting model at each issue time t_0. The system uses a strict data cut-off at t_0, meaning that only information available by that time can be used to generate a forecast. This provides a practical basis for the operational setup described in Section 5.

A17 Rain forecast. I do not assume that an external rainfall forecast will always be available. If one is provided, I assume that it contains ten-minute rainfall totals for R1 and R2 over the future intervals $t_0 + \Delta, \ldots, t_0 + H\Delta$. Since future rainfall information is not guaranteed (F7), I treat it as an optional input with its own availability flag. I also train the model to generate forecasts when this information is unavailable.

A18 Warning levels. I assume that official warning thresholds are not available because they are not specified in the assignment. Instead, I define high water at each station as a water level at or above its 90th percentile, calculated using the training data. This provides a simple station-specific way to identify unusually high levels and determine the high-water weighting in the training loss. If official warning thresholds become available, I replace these percentile-based thresholds with the official values (Q8).

A19 Error distribution. I initially assume that forecast errors at each station and prediction lead can be approximated by a Gaussian distribution. This provides a straightforward starting point for modelling uncertainty in continuous water-level predictions. Based on this assumption, the forecast head predicts both a mean and a standard deviation and is trained using Gaussian negative log-likelihood. I then check prediction interval coverage across different flow conditions. If the intervals do not provide reliable coverage, particularly during peak water levels, I consider switching to a quantile-based loss (U9).

1.3 Forecasting task and data specification

1.3.1 Task

- **When a forecast is made.** A new forecast is made every ten minutes, day and night. I call the moment it is made the issue time t_0.
- **What it may use.** Only readings that had actually arrived by t_0, and dam release plans that had been announced by t_0. Nothing from the future.
- **How far back it looks.** The last three hours, split into $L = 18$ ten-minute steps: the interval $(t_0 - 180 \text{ min}, t_0]$.
- **How far ahead it predicts.** One hour, in $H = 6$ ten-minute steps. Step $k = 1, \ldots, 6$ is the time $t_0 + 10k$ minutes. The service only needs the one-hour value ($k = 6$), but I predict all six: operators can then see whether the level is rising or falling, and each training example gives six targets to learn from instead of one.
- **What it outputs.** For each of the four stations S1–S4 and each step k, a best guess $\hat{y}_{s,k}$ of the water level and an uncertainty $\hat{\sigma}_{s,k}$ (a standard deviation). These are reported as the best guess with a 90% range, $\hat{y} \pm 1.645\,\hat{\sigma}$, which assumes the errors are roughly normal. The tributary station T1 is used as an input only; it is not forecast.

1.3.2 How far back the model needs to look

The model must see far enough back to catch anything that can still reach a station within the next hour. Two kinds of delay matter: the time water takes to flow down the river (A11), and the time rain takes to reach the river (A12). I size the window for the slowest case, the slowest wave on the longest route, and add a margin m, so the model sees the start of a rise and not just its tail:

$$\underbrace{L\,\Delta}_{\text{window length}} \;\geq\; \max\Big(\underbrace{\tau_{\text{Dam}\to\text{S4}}}_{\text{dam water}},\; \underbrace{\tau_{\text{rain}} + \tau_{\text{S2}\to\text{S4}}}_{\text{rain water}}\Big) + m, \qquad \text{travel time } \tau_{\text{route}} = \frac{\text{distance } D_{\text{route}}}{\text{slowest speed } c_{\min}}.$$

With a slowest speed of $c_{\min} = 1.2$ m/s, rain reaching the river within one hour, and a 30-minute margin, three hours ($L = 18$) is enough for the distances allowed in A11. If the real distances are longer, the same rule gives a longer window: for example, 15 km from the dam to S4 takes about 3.5 hours to travel, so it needs a four-hour window ($L = 24$).

I also check the window on the data instead of trusting the rule alone. For each upstream station I measure how long its changes take to show up downstream, separately for calm

periods and floods, and I compare windows of 12, 18, 24 and 36 steps on the validation years. When unsure, I choose the longer window, because a window that is too short makes the model see floods too late.

1.3.3 Inputs and outputs

Each training example is one issue time. During training, many issue times from the history are processed together as a batch of size B; in daily use, $B = 1$. The six locations ("nodes") are always in the order Dam, S1, T1, S2, S3, S4. The model's main input X is a grid: for each time step and each location, nine numbers.

Name	Shape	What it holds	Known at t_0?
Inputs			
X	$B \times 24 \times 6 \times 9$	24 time steps × 6 locations × 9 values. Steps 1–18 are the past three hours (up to t_0); steps 19–24 are the next hour	Past: yes, with gaps marked. Next hour: only the dam's release plan and a rain forecast, if one is supplied
c	$B \times 24 \times 4$	Time of day and day of year for each step, as sine and cosine, so that 23:50 sits next to 00:00	Yes
$\tilde{y}$	$B \times 4$	The most recent valid level at each station S1–S4	Yes
a	$B \times 4$	How old that reading is, in minutes	Yes
A	6×6	Which location flows into which (1 = connected)	Yes; never changes
e	5	The length of each of the 5 river links, in km	Yes; never changes
M	5×2	Which rain gauge drains into which station	Yes; never changes
Outputs			
$\hat{Y}$	$B \times 4 \times 6$	Best-guess level at each station, for each of the next six steps	Produced at t_0
$\hat{\Sigma}$	$B \times 4 \times 6$	The uncertainty of each guess	Produced at t_0
Training only			
Y	$B \times 4 \times 6$	The levels that were actually measured	No: these are the answers

Name	Shape	What it holds	Known at t_0?
Ω	$B \times 4 \times 6$	1 if that answer was actually measured, 0 if it is missing, so missing answers are left out of the loss	No

The nine values per location and step are: (1) water level, (2) whether the level was actually measured, (3) how old the level reading is, (4) rain, (5) whether the rain was actually measured, (6) whether the rain value is a forecast, (7) dam release, (8) whether the release is a plan rather than a record, and (9) whether the release had to be filled in. A value that does not apply to a location is set to zero, along with its flag: the dam has no level or rain, and the stations have no release.

1.3.4 Putting all sources on one timeline

The data arrive on different clocks: levels every ten minutes, rain at irregular times, and releases whenever a gate moves. Before the model sees anything, a fixed procedure (with nothing learned) puts everything on the same ten-minute grid:

1. **Make the time steps.** Step τ ends at $t_\tau = t_0 + (\tau - L)\Delta$ and covers the ten minutes before that, $(t_\tau - \Delta,\, t_\tau]$. So step $\tau = L$ is "now".
2. **Remove anything from the future.** Drop every reading that arrived after t_0, and every release plan announced after t_0. If two plans cover the same time, keep the one announced last (A3, A7).
3. **Remove bad readings.** Readings that are out of range, stuck at one value, or jump suddenly are treated as missing (A4). This happens before the next steps, so a late or faulty reading can never be picked for a time step.
4. **Levels** (5 stations, 3 values each). Each step takes the latest valid reading in its ten minutes, plus whether one was measured and how old it is. Future steps are left empty and marked as not measured.
5. **Rain** (2 gauges, 3 values each). A rain report covering several steps is shared out over the steps it overlaps (A5). Each step gets the rain amount, whether it was measured, and whether it is a forecast. Future steps hold the external rain forecast, if there is one.
6. **Move rain from gauges to stations.** The map M assigns gauge R1 to station S2 and gauge R2 to station S3 (A13, A14). After this, rain is listed by station, just like the levels.
7. **Releases** (the dam, 3 values). For each step, the average release over its ten minutes, weighted by how long each gate setting lasted: the recorded release in the past, the latest announced plan in the future (A7). Each step also records whether the value is a plan and whether it had to be filled in.

8. **Combine the values.** Each station gets its 3 level values, its 3 rain values and 3 zeros for release (9 values). The dam gets 6 zeros for level and rain, plus its 3 release values (9 values).
9. **Stack the locations.** Put the dam first, then the five stations. This gives the input grid X of shape $B \times 24 \times 6 \times 9$.
10. **Rescale.** Levels are shifted and scaled per station, using each station's median and typical spread β_s (A6). Rain becomes $\log(1 + \text{rain})$, so heavy storms do not dominate. Releases are divided by a typical release, and reading ages by three hours. All of these scales are computed from the training years only.

The stations are not evenly spaced along the river. This needs no special handling here: the distances are given to the model later, as the lengths of the river links.

1.3.5 What the model is allowed to know at issue time

- **Known:** past levels at S1–S4 and T1 and past rain at R1 and R2, if they arrived by t_0; recorded dam releases; release plans announced by t_0; the river map, link lengths and rain-gauge map; the date and time; and an external rain forecast, if one is supplied.
- **Not known:** any level after t_0; any rain after t_0 (apart from a supplied forecast); readings that are late and have not arrived yet; plans announced after t_0; and what the dam will actually release after t_0.
- **In training:** every training example rebuilds exactly this view for its own issue time, using the recorded arrival and announcement times. A value that arrived later is never filled in after the fact, so the model never learns from information it will not have in real use.

1.4 Model computations and design justification

1.4.1 Notation

Symbol	Meaning
$t_0,\ \Delta$	issuance time; step of 10 min
L, H, T	lookback (18), horizon (6) and grid length $T = L + H = 24$
τ	grid column $1, \ldots, T$; $\tau = L$ is t_0
i, j	nodes (Dam, S1, T1, S2, S3, S4); s indexes the stations S1–S4
$\mathcal{N}_{\text{in}}(i)$	upstream neighbours of i: Dam→S1; S1, T1→S2; S2→S3; S3→S4
d_{ji}	along-river distance of edge $j \to i$
$x_{i,\tau} \in \mathbb{R}^9$	the nine channels of node i at column τ
$z_{i,\tau},\ h_{i,\tau}^{(\ell)},\ s_i^{(k)} \in \mathbb{R}^{64}$	projected input; output of T block ℓ; node summary after graph layer k
ℓ	T block index $1, \ldots, 4$, with dilation $r_\ell = 2^{\ell-1}$

k	graph layer $1, \ldots, K$ in Block 3 ($K = 4$); lead time $1, \ldots, H$ in Block 4
$E_i,\ P_\tau,\ u_k$	learned node, column and lead embeddings
$\tilde{y}_s,\ \beta_s$	last valid level of station s; its normalisation scale
$\hat{y}_{s,k},\ \hat{\sigma}_{s,k}$	forecast mean and standard deviation
$\Omega_{s,k},\ w_{s,k}$	target observed mask; high-water loss weight

1.4.2 Block 1: per-source input projection

The dam row carries a flow in m^3/s with schedule flags; the station rows carry levels in metres and rain. Each kind gets its own matrix, so that after this block every row "speaks the same language":

$$\bar{z}_{i,\tau} = \begin{cases} W_{\text{dam}}\, x_{i,\tau} + b_{\text{dam}}, & i = \text{Dam}, \\ W_{\text{gauge}}\, x_{i,\tau} + b_{\text{gauge}}, & i \in \{\text{S1}, \text{T1}, \text{S2}, \text{S3}, \text{S4}\}, \end{cases} \qquad W_{\text{dam}}, W_{\text{gauge}} \in \mathbb{R}^{64\times 9},$$

$$z_{i,\tau} = \bar{z}_{i,\tau} + E_i + P_\tau + W_c\, c_\tau.$$

In: $X\,[B, T, 6, 9]$, $c\,[B, T, 4]$. Out: $Z\,[B, T, 6, 64]$.

In practice X is split on the node axis (dam row and five station rows), each part is projected, and the parts are concatenated back. W_{gauge} is shared by the five stations, so the node embedding E_i tells later blocks which station they are reading. The column embedding P_τ tells them which columns are past and which are future, and the calendar term carries daily and seasonal patterns (A15).

1.4.3 Block 2: temporal encoder f_{time} (four dilated causal T blocks)

$$h^{(0)}_{i,\tau} = z_{i,\tau}, \qquad u^{(\ell)}_{i,\tau} = \sum_{m=0}^{2} W^{(\ell)}_m\, h^{(\ell-1)}_{i,\,\tau - m r_\ell} + b^{(\ell)}, \qquad r_\ell = 2^{\ell-1} \in \{1, 2, 4, 8\},$$

$$h^{(\ell)}_{i,\tau} = h^{(\ell-1)}_{i,\tau} + \mathrm{GELU}\big(\mathrm{LN}(u^{(\ell)}_{i,\tau})\big), \quad \ell = 1, \ldots, 4; \qquad s^{(0)}_i = h^{(4)}_{i,T}.$$

In: $Z\,[B, T, 6, 64]$, reshaped to $[B{\cdot}6,\ 64,\ T]$ for the convolution and reshaped back. Out: $S^{(0)}\,[B, 6, 64]$. Each block mixes three columns of the same node: now, r_ℓ back and $2r_\ell$ back. Columns before $\tau = 1$ are zero, so the convolution is causal. Block ℓ looks $2r_\ell$ columns back, so the stack reaches $1 + 2(1 + 2 + 4 + 8) = 31 \geq T$ columns, and the last column of each node has seen the whole grid, including the dam's announced schedule in the future columns. That last column is the node's summary.

Why this block. The reach is exact and easy to check; undilated blocks would need 12 layers to cover 24 columns. One set of weights serves all six nodes, which gives six times more data per parameter, important when floods are rare (A1). A GRU was considered: it also covers the window, but it squeezes history into one vector step by step, older information fades, and its reach cannot be stated exactly. Missing level cells arrive as zeros next to their observed flag and age, so the kernels learn to discount them.

1.4.4 Block 3: directed message passing f_{graph} (repeated $k = 1, \ldots, K$, $K = 4$)

$$\mathrm{msg}_{j\to i} = W^{(k)}_M \big[\, s^{(k-1)}_j;\ \phi(d_{ji}) \,\big], \qquad s^{(k)}_i = \mathrm{LN}\Big(s^{(k-1)}_i + \mathrm{MLP}^{(k)}\big[\, s^{(k-1)}_i;\ \textstyle\sum_{j\in\mathcal{N}_{\text{in}}(i)} \mathrm{msg}_{j\to i} \,\big] \Big).$$

In: $S^{(k-1)}\,[B, 6, 64]$, adjacency $A\,[6, 6]$, distances $e\,[5]$. Out: $S^{(k)}\,[B, 6, 64]$. ϕ is a small MLP that turns a distance into features.

- **Direction and confluences are physics, not learned.** Messages flow only along downstream edges (A9), and incoming messages are summed, mirroring flows that add at a confluence (A10). The dam and T1 have no upstream neighbours, so only the residual path updates them.
- **Travel-related patterns are learned from historical data** Each message carries the sender's summary, which Block 2 built from the sender's whole window. It records when and how fast that station's level changed (for example, "S1 started rising 40 minutes ago, fast"). The MLP learns what that history means for the receiver's next hour, which differs between floods (fast travel) and low flow (slow travel) (A11). Distance enters each message as a physical cue.
- **Depth.** After layer k, a node holds information from every upstream node within k hops. Dam→S1→S2→S3→S4 is four hops, hence $K = 4$.
- **Why time before space.** A message is only useful if it carries the sender's recent history, not a single instant, so the temporal encoder runs first. Each station also handles its own gaps with its own flags before sharing, so one outage does not spread downstream. The cost is that history is never mixed across stations before the graph; alternating temporal and graph layers is the fallback if validation shows such errors.
- **Why plain message passing rather than attention.** Attention could let a node look back directly at the column of its upstream neighbour that matches the current travel time. I keep the simpler layer: it matches the course baseline, has fewer parameters to learn from few flood events, and the 31-column reach already lets each summary encode timing. If forecast peaks arrive at the wrong time in validation, attention over the upstream columns is the first extension to try.

1.4.5 Block 4: direct forecast head g

$$\big[\mu_{s,k},\ \rho_{s,k}\big] = W_2\,\mathrm{ReLU}\big(W_1\,[\,s_s^{(K)};\ u_k\,] + b_1\big) + b_2, \qquad k = 1, \ldots, H,$$
$$\hat{y}_{s,k} = \tilde{y}_s + \beta_s\,\mu_{s,k}, \qquad \hat{\sigma}_{s,k} = \beta_s\big(\mathrm{softplus}(\rho_{s,k}) + \varepsilon\big).$$

In: the four station summaries $S^{(K)}\,[B, 4, 64]$ (the dam and T1 rows are dropped) and $\tilde{y}\,[B, 4]$. Out: $\hat{Y},\ \hat{\Sigma}\,[B, 4, H]$.

At ten to sixty minutes ahead, "the level stays where it is" is already a strong forecast, so the head only learns the change from $\tilde{y}$. The softplus keeps $\hat{\sigma}$ positive.

The forecast is direct, not autoregressive:

	Direct (chosen)	**Autoregressive**
Error build-up	none; every lead comes from the same inputs	each step's error feeds the next
Future inputs needed	none beyond the grid	its own predicted levels, and future rain that is unknown

Dam schedule	the whole announced schedule is in the input at once	must be fed step by step
Cost	one forward pass	six passes
Weakness	leads can be less smooth; the shared head and lead embedding limit this	—

1.4.6 Training objective

$$\mathcal{L} = \frac{\sum_{b,s,k} \Omega_{b,s,k}\, w_{b,s,k} \Big[\log \hat{\sigma}_{b,s,k} + \dfrac{(y_{b,s,k} - \hat{y}_{b,s,k})^2}{2\,\hat{\sigma}^2_{b,s,k}} \Big]}{\sum_{b,s,k} \Omega_{b,s,k}\, w_{b,s,k}}, \qquad w_{b,s,k} = 1 + \lambda\, \mathbf{1}\big[\, y_{b,s,k} \geq q_s^{0.9} \,\big].$$

- **Gaussian negative log-likelihood** trains the mean and the spread together (A19), so the model reports wider intervals where it is less sure, for example at S3 and S4 during rain on the unmonitored catchment. A single-number loss such as MAE or MSE would settle on a central value when information is missing and would under-predict floods.
- **The mask** Ω removes targets that were never measured, so gaps do not count as errors.
- **The weight** w counts high-water steps $1 + \lambda$ times, because a missed flood costs far more than an error at low flow. $q_s^{0.9}$ is station s's 90th-percentile level in the training years (A18), and $\lambda = 4$ is tuned on validation.
- **Procedure.** One sample per historical t_0, rebuilt exactly as it was available at t_0 (S3). Training adds random gaps (recent readings dropped with probability 0.1), simulated reporting delays (A3) and a hidden rain forecast (probability 0.5), so the model has seen the faults it will meet. Adam optimiser and early stopping on validation loss.
- **Split and evaluation.** Train, validation and test are split chronologically by year, with a gap of at least T columns between them, and validation and test must each contain real flood events. I report MAE and RMSE per station and lead, the same on high-water steps only, mean bias (to catch systematic under-prediction), peak-height and peak-timing error per flood event, and coverage of the 90% interval. All are compared with persistence ($\tilde{y}$ repeated) and a linear model on lagged inputs, and with ablations of our model without the graph and without the dam schedule.

1.4.7 How each source reaches each forecast

Station	Main drivers	Path in the model
S1	dam releases, recorded and planned; S1's own history	edge Dam→S1 (one hop). A release change planned within the hour reaches S1 through the dam's future columns.

S2	S1 and the monitored tributary T1; R1 runoff; the dam through S1	edges S1→S2 and T1→S2, summed as a confluence (A10); R1 in S2's own cells (A13); the dam at two hops.
S3	S2; the unmonitored tributary via R2; S3's own history	edge S2→S3; R2 in S3's own cells (A14); what R2 cannot explain shows up as a wider $\hat{\sigma}$.
S4	S3 and everything upstream of it	edge S3→S4; the dam at four hops, T1 and R1 at three, R2 at two.

1.4.8 Physical assumptions versus learned mappings

Fixed by physical assumption	**Learned from data**
influence flows downstream only (A9)	each station's level–flow relation (rating curve)
network shape and confluence points (F4, F5)	the travel delay and how it changes with flow
flows add at confluences, so messages are summed (A10)	how flood waves flatten as they travel
distances enter as edge inputs (F4, A8)	how rain turns into runoff, and its lag
where each rain gauge drains (A13, A14)	the unmonitored tributary's baseflow, implicitly
no information after t_0 except announced schedules (F3, F7)	how actual releases tend to differ from schedules
the lookback bound from the slowest delay (A11, A12)	the size of the uncertainty per station and lead

Treatment of physical relationships and uncertainty. I do not use a separate physical simulator. Physics enters the learned model as structure: directed edges, summation at confluences, distances on edges, the drainage map, causal convolutions and the information cut-off at t_0. Uncertainty is handled inside the model through the Gaussian output, trained with the likelihood above, and checked through interval coverage.

Size. Hidden width 64, $K = 4$; roughly 1.8×10^5 parameters (T blocks $\approx$ 49k, graph layers $\approx$ 120k, projection and head $\approx$ 13k).

1.5 Operation and exceptional conditions

Each cycle rebuilds the grid at t_0, runs one forward pass and publishes the forecast with quality flags. Every fault below changes only the inputs and flags, never the code path.

1.5.1 (i) Missing observations

- A reading is missing if none falls in its column or quality control rejects it (A4). The cell gets value 0 with observed = 0, and the age channel keeps counting. The T blocks see the flag and the age next to the zero, so a filler is never read as a real level or passed downstream as one.

- If a station's latest reading is missing, $\tilde{y}_s$ is its last valid reading and the age is part of the input. Training with random gaps teaches the model to lean on neighbours (for example S2 on S1 and T1) and to widen $\hat{\sigma}$.
- If a station has no valid reading in the whole lookback, its forecast is marked degraded (using the age a) and operators are alerted. Downstream stations still forecast from their other inputs.
- Missing rain gets rain observed = 0; the model has met such gaps in training.
- If the model or the data pipeline fails, the service publishes a persistence forecast ($\tilde{y}$ repeated) with a flag, so the authority is never left without a forecast.

1.5.2 (ii) Delayed observations

- At t_0, a reading that has not arrived is treated as missing, as in (i). Its age tells the model how stale the last value is.
- When it arrives, it is stored with its arrival time, and the next cycle ten minutes later uses it. Issued forecasts are not rewritten; the next forecast replaces them.
- In training, availability follows the logged arrival times if the authority has them (Q6), otherwise delays are simulated from A3. A late value is never back-filled into an earlier sample.

1.5.3 (iii) Announced future releases

- Each future dam column takes the latest schedule announced by t_0 whose effective period covers it, with release planned = 1. A schedule announced at 09:20 is ignored by the 09:00 and 09:10 forecasts and first used at 09:20.
- The newest announcement replaces earlier ones for the steps it covers (A7). A future step that no schedule covers repeats the last release with release imputed = 1, and the model learns to trust it less.
- Because the T-block reach covers all T columns, the dam's summary includes the planned releases, and Block 3 carries them to S1 and downstream.
- In training, future dam cells are rebuilt from the announcement log as known at each historical t_0, never filled with the releases that actually happened.

1.5.4 (iv) Differences between scheduled and actual releases

- Past columns always hold the recorded release (planned = 0), so a deviation from the plan appears in the next cycle and the following forecast corrects itself within ten minutes.
- Training pairs as-announced schedules with what actually happened, so the model learns how far operations usually drift from plans (for example, late starts) and widens $\hat{\sigma}$ accordingly.

- Each cycle compares planned and recorded releases for the latest steps. If they differ by more than 20% of the planned flow (a chosen threshold), the forecast carries a dam deviation flag and operators are notified.
- Real-time gate telemetry (Q7) would shorten the time before a deviation is seen.

1.5.5 Running every ten minutes is feasible

Step	Work per cycle	Expected time
assemble data	query three hours of five level gauges, two rain gauges and the dam log; filter, bin and normalise about 1,000–2,000 records	under 5 s
forward pass	about 10^7 multiply-adds: four T blocks over 6×24 cells and four graph layers over five edges	under 50 ms on one CPU core
publish	write forecasts and flags	under 1 s

A cycle takes well under a minute against a ten-minute budget, on one ordinary server (A16). Retraining runs offline, for example monthly: three years give about 158,000 issuance times, which this model trains on within an hour. Each forecast is later compared with what was observed; if interval coverage drifts, retraining is triggered (A15).

1.6 Uncertainty, limitations and additional information

The largest uncertainty is the unmonitored tributary, which joins just above S3 and affects S4, the station that protects the town. A gauge on it (Q1) is the single most valuable request. The system still runs without it, with wider intervals at S3 and S4.

1.6.1 Uncertainties and limitations

#	Issue	Effect on forecast	Our response	Still unresolved
U1	unmonitored tributary between S2 and S3	S3 and S4 errors grow during rain on its catchment; baseflow changes are invisible	R2 as a proxy (A14) plus S3's own history; the model learns wider $\hat{\sigma}$ at S3 and S4	rain away from R2, wet versus dry soil, slow baseflow drift
U2	travel times change with flow	a wrong delay makes the forecast peak too early or too late	T-block summaries record when and how fast levels changed upstream, so the graph learns flow-dependent delays; L is sized for the slowest case (S3)	waves slower than A11 assumes would fall outside L; monitored, and L raised if seen; attention is the next step if peak timing is poor

#	Issue	Effect on forecast	Our response	Still unresolved
U3	rare extreme floods	few training examples; peaks may be under-predicted; record levels need extrapolation; gauges may saturate	high-water loss weight; NLL widens intervals; QC for saturation; persistence fallback	a flood larger than any in the record cannot be learned from data
U4	rain after t_0, and rain between the two gauges	rain falling now, or a storm between R1 and R2, is missed	the catchment lag (A12) limits its effect within an hour; optional external forecast (Q4)	small convective storms between gauges
U5	dam operations differ from the schedule	S1, then downstream stations, are mis-forecast until the recorded release arrives	recorded release used next cycle; deviation flag; learned caution (§5)	an unannounced emergency release is unseen for 10–20 min
U6	missing or delayed data	fewer inputs, larger errors	masks, age, training with gaps, degraded flags (§5)	a long outage at S3 or S4 leaves the town-side forecast weak
U7	changes in the river or the dam rules	forecasts become biased	coverage monitoring triggers retraining (A15)	an abrupt change between retrainings
U8	backwater near confluences	small level effects upstream of a confluence are not modelled	neglected (A9); reverse edges can be added if data show the effect	—
U9	Gaussian error shape	errors on rising limbs may be skewed, so intervals are too narrow at peaks	coverage checked per flow regime; switch to a quantile loss if it fails (A19)	—

1.6.2 Additional information requested

None of these is required for the system to run; each one would just improve it.

#	Request	How it helps	Usable without it?
Q1	a level or flow gauge on the unmonitored tributary	turns the largest blind spot into a measured node feeding S3	yes: R2 proxy, wider intervals at S3 and S4
Q2	rating curves or discharge measurements at S1–S4 and T1	flow inputs make the confluence sum physically exact and improve extrapolation in floods	yes: the level–flow relation is learned
Q3	observed travel times, or a channel survey, for each reach	confirms $c_{\min}$ and the lookback L	yes: L from A11, checked by cross-correlation
Q4	an external rainfall forecast or radar nowcast	improves rises driven by rain falling now	yes: trained with it hidden half the time
Q5	more rain gauges or radar rainfall, mainly over the unmonitored catchment and between the dam and S2	a better spatial picture of rain	yes: two point gauges
Q6	data arrival timestamps, maintenance logs and gauge type	exact delay simulation in training; better QC; confirms what a rain reading means (A5)	yes: simulated delays (A3)
Q7	real-time dam gate telemetry and the full schedule revision log	faster detection of deviations; exact as-of schedules for training	yes: recorded releases, ten-minute lag
Q8	official warning levels per station	loss weights and alerts in the authority's own terms	yes: 90th-percentile proxy (A18)
Q9	reservoir level and spillway rules	anticipates uncontrolled spill when the reservoir is full	yes, with higher risk in extreme events

2 Question 2: Motion or Memory?

2.1 Setup and checks

Videos were split by dataset index (0–7,999 training, 8,000–8,999 validation, 9,000–9,999 test). All experiments ran on one NVIDIA RTX PRO 6000 Blackwell Server Edition GPU. Training time is the mean wall-clock time per epoch over all 30 epochs, excluding the dataset download and the validation check run after each epoch. Inference time is the time to predict all 1,000 original test videos ($1\times$, $t = 1$) at batch size 32, timing only the forward passes after each batch is on the GPU, after one untimed warm-up batch. Table 1 confirms that the transformation and window functions used in every experiment behave as defined.

Table 1: Checks of the transformation and window functions (synthetic tensors).

Check	Expected	Yours
(a) $\max \lvert M(M(x)) - x \rvert$	0	0 (`torch.equal`: True)
(b) $\max \lvert R^4(x) - x \rvert$	0	0 (`torch.equal`: True)
(c) $\max \lvert MRM(x) - R^{-1}(x) \rvert$	0	0 (`torch.equal`: True)
(d) bright pixel of $M(p)$ (frame, row, column)	(7, 10, 43)	(7, 10, 43)
(e) bright pixel of $R(p)$ (frame, row, column)	(7, 43, 10)	(7, 43, 10)
(f) 1× input / target	1–5 / 6–10	1–5 / 6–10
(g) 2× input / target	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19	1, 3, 5, 7, 9 / 11, 13, 15, 1
(h) 3× input / target	1, 4, 7, 10, 13 / 16, 19	1, 4, 7, 10, 13 / 16, 19

Additional checks (not required): every $g \in G$ is exactly undone by its inverse, the eight elements of G produce eight different images, and the allowed start frames match the handout for both training rules.

2.2 Q2.1

Mirroring changes a flow direction θ to $180° - \theta$, while a 90° anticlockwise rotation changes it to $\theta + 90°$. If all directions were equally common, these transformations would leave the direction distribution unchanged. This would suggest that the training data already contains motion in all directions, although it would not guarantee that the model can handle transformed digit appearances.

Figure 1(a) is not perfectly uniform: four of the eighteen bins have 95% confidence intervals that do not include the 1/18 reference line. Randomisation tests on the 50 videos give $p = 0.055$ for mirroring and $p = 0.080$ for rotation, so the evidence of asymmetry is inconclusive at the 5% significance level.

As supplementary analysis, I repeated the tests using all 8,000 training videos. Both transformations show statistically significant differences ($p < 0.001$), with TV distances of 0.071 for mirroring and 0.079 for rotation. However, the imbalance is relatively mild: each 20° bin contains between 0.72 and 1.21 times its expected share under a uniform distribution. Thus, motion in all directions is represented in the training data, but not equally often.

The median optical-flow speed is 3.32 px/frame in Figure 1(b). This corresponds to approximately 6.64 px between observations at 2× speed and 9.96 px at 3×, using the rounded median. The peaks in the speed histogram are consistent with digits moving by near-integer pixel displacements; about three-quarters of the measured speeds is within 0.25 px/frame of an integer.

Optical flow can be unreliable where the digits overlap, and about one-third of frame pairs contain some overlap. Excluding these pairs changes the median speed by only 0.04 px/frame and the direction distribution by a TV distance of 0.028. This suggests that overlap has little effect on the main conclusions, provided the observed direction patterns and speed estimates remain similar.

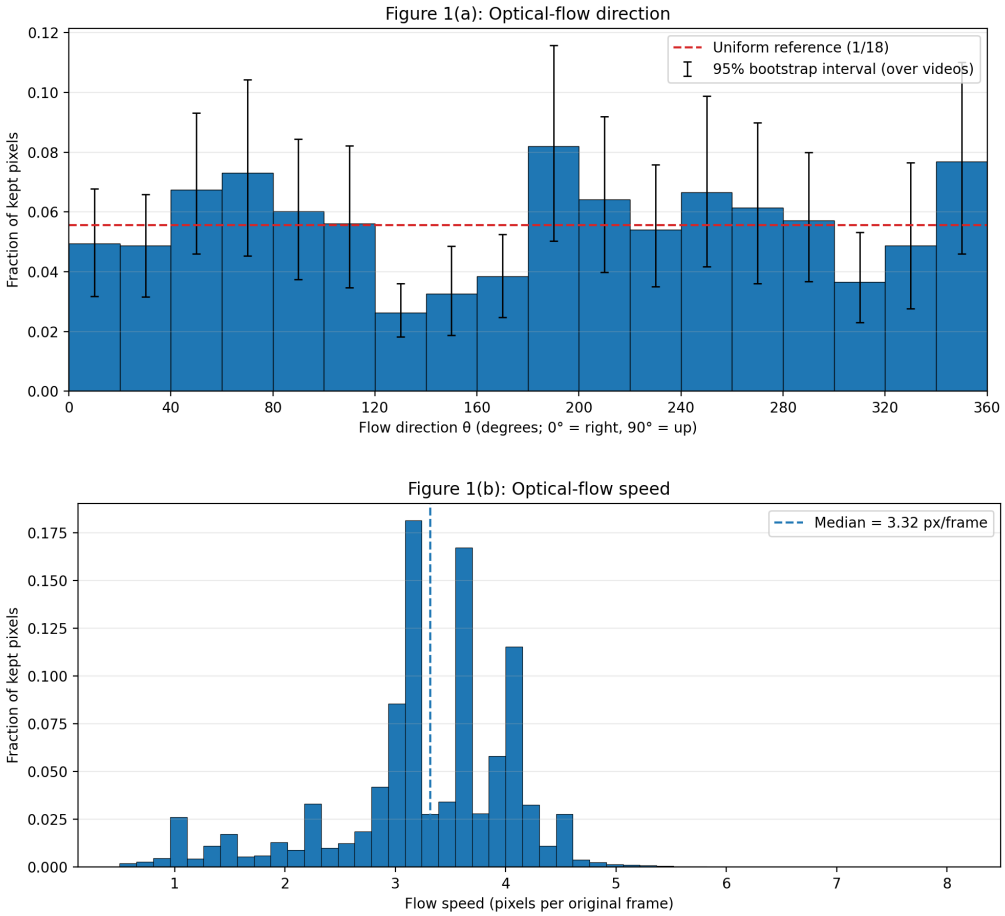


Figure 1: Optical-flow statistics on training videos 0–49 (950 frame pairs, 103,050 kept pixels), as fractions of kept pixels. (a) Top: flow direction θ (0° = right, 90° = up), with 95% bootstrap intervals over videos; dashed line = 1/18. (b) Bottom: flow speed; median 3.32 px/frame (dashed line).

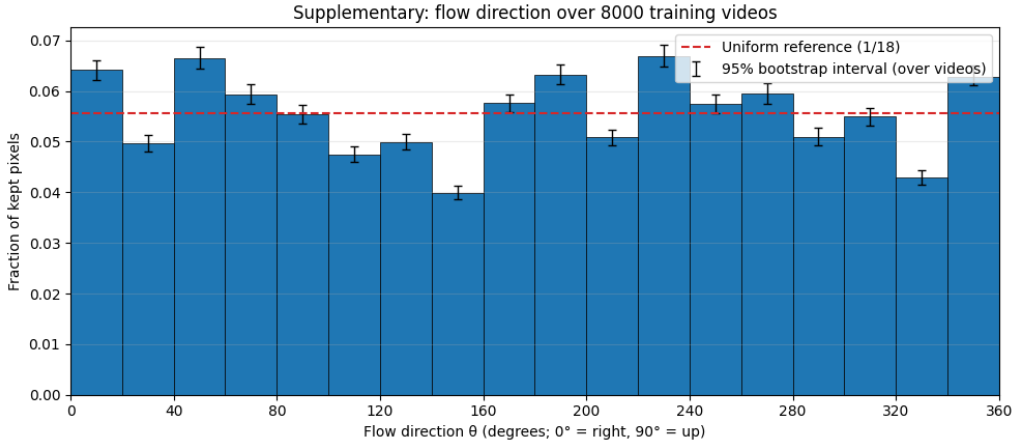


Figure S1: Supplementary (not Figure 1): flow direction over all 8,000 training videos (16,443,567 kept pixels), with 95% bootstrap intervals over videos; dashed line = 1/18.

2.3 Q2.2

Transformation	Motion	Appearance
Mirror	Changes a little: it swaps the common and the rare diagonal (TV 0.071)	Changes: most mirrored digits are no longer real digits
Rotation	Changes a little: the same swap (TV 0.079)	Changes a lot: rotated digits look like nothing in training
Speed-up	Changes: digits jump about 6.64 px (2×) or 9.95 px (3×) between the frames the model sees	Does not change: every frame is a real, untouched frame

Each transformation mostly changed one thing and leave the other nearly alone which is what makes them useful together. Speed-up changes only the motion, so if the model fails there, the digits' appearance cannot be the reason. Mirror and rotation change the motion by about the same small amount, but rotation changes the appearance much more. So if the model does clearly worse on rotated videos than on mirrored ones, the extra difficulty comes from appearance, not motion. Mirror and rotation are not pure appearance tests, though, because they do shift the motion slightly, and I keep that in mind when reading the results. All three transformations are also exact and can be undone. The correct future of a transformed video is simply the transformed true future, $T(Y)$, so I can measure how well the model does on transformed videos without needing any new data.

Table 2: Test error at 1× speed, $t = 1$. Model A: mean (seed 0, seed 1). Model A's mean training time is 1.55 s per epoch (1.61, 1.48).

	Model A	Blank	Copy-last
MSE_1	0.0170 (0.0171, 0.0170)	0.0426	0.0492
MSE_2	0.0205 (0.0208, 0.0201)	0.0426	0.0604
MSE_3	0.0227 (0.0232, 0.0223)	0.0426	0.0669
MSE_4	0.0245 (0.0250, 0.0239)	0.0425	0.0708
MSE_5	0.0261 (0.0267, 0.0256)	0.0425	0.0732
$E^{(5)}$	0.0222 (0.0225, 0.0218)	0.0426	0.0641
Parameters	480,784	0	0



Figure 2: Model A (seed 0) on three test videos at 1×, $t = 1$. Frames 1–5 are the input; in frames 6–10 the upper row is the true future and the lower row is model A's forecast. Video 9000 has overlapping digits.

2.4 Q2.3

Model A never operates directly on the 64×64 image. Pixel-unshuffle folds each 4×4 patch into one cell of a 16×16 grid, so each grid cell represents 4×4 original pixels. The 3×3 convolutions are the only operations that expand the spatial receptive field; each can extend it by one cell in every direction. The 1×1 output head, pixel shuffle, and element-wise gate and cell-state updates do not expand it.

Within one time step, new input information passes through the convolutions in both ConvLSTM layers and can influence positions up to two cells away. Layer 1's previous hidden state also passes through both layers' convolutions, whereas layer 2's previous hidden state passes through only the second layer's convolution. The corresponding spatial influence extends by up to two cells and one cell, respectively. Cell-state updates are element-wise and do not add spatial movement.

Therefore, the longest current-input path spans two cells, equivalent to 8 pixels horizontally or vertically in the original image or approximately 11.3 pixels diagonally. This describes the spatial reach within one time step along that path, not a strict limit on motion between frames, because recurrent hidden and cell states can propagate information farther across multiple time steps.

The maximum offset along either axis is $2 \times 4 = 8$ pixels. The diagonal offset is therefore

$$d = \sqrt{0} = 8\sqrt{2} \approx 11.3.$$

2.5 Q2.4

Model A is clearly better than both reference predictors (Table 2). Its average error $E^{(5)}$ is 0.0222, about half that of Blank (0.0426) and a third of Copy-last (0.0641), and it wins at every horizon. The two seeds agree closely (0.0225 and 0.0218).

Its error grows steadily with the horizon, from 0.0170 at the first predicted frame to 0.0261 at the fifth, about 50% worse. This is expected: from the second prediction onwards the model works from its own earlier, slightly wrong output, so mistakes build up.

Blank beating Copy-last looks strange but has a simple cause. Under squared error, a digit drawn in the wrong place is penalised twice: once for bright pixels where there should be none, and once for the dark pixels where the digit really is. Blank only pays the second cost. By the fifth horizon Copy-last's error (0.0732) is close to double Blank's (0.0425).

The same effect explains why model A's predictions blur (Figure 2). When it is unsure where a digit will be, a soft grey smear over the likely area costs less than a sharp digit in the wrong place. The digits keep roughly the right position for the first frames, but lose their shape by about frame 8, and in later frames a faint blocky texture from the 4×4 patch folding shows through. Thin digits suffer most: in video 9012 the 7 fades to a faint smudge while the 4 is still tracked as a blob.

Overlap is where the model fails most clearly. In video 9000 the 5 and the 8 start out overlapping and then move apart, the 5 upwards and the 8 down-left. Model A never separates them. It keeps a single blob and stretches it vertically, as if hedging between the two directions, instead of tracking two separate digits.

Finally, the validation error was still falling at epoch 30 for both seeds, so model A has not fully converged under the fixed 30-epoch budget. Its numbers should be read with that in mind.

2.6 Q2.5

Mirror. I expect model A to mostly pass. Mirroring shifts the motion statistics only slightly (TV 0.071, Q2.1). Mirrored digits are not real digits, but they are made of the same kind of strokes the model has seen, and some digits (0, 1, 8) barely changed. I expect E_M to be only a little above $E^{(5)}$, within about 15%, but C_M to be clearly above zero, because nothing in the model forces it to treat a mirrored video the same way as the original.

Rotation. I expect rotation to be harder than mirroring. It shifts the motion by about the same amount (TV 0.079), but it turns vertical strokes into horizontal ones, a much bigger change in appearance. I expect E_R and C_R to be noticeably higher than E_M and C_M, perhaps 15–30% above $E^{(5)}$, with digits losing their shape sooner. I expect the model to still move the digits in roughly the right direction.

Speed. Model A's reach is about 8 px per axis in one step (Q2.3). At $1\times$ the typical jumps between frames are at most 4 px per axis, well within reach. At $2\times$ they reach about 8 px, right at the limit, and the model has never seen motion this fast, so I expect clearly worse predictions, with the digits still roughly tracked. At $3\times$ the jumps are 9–12 px along at least one axis, beyond the reach, so I expect model A to break down there: digits that lag behind, stall or fade into a blur, with $E^{(2)}$ close to or worse than

Blank's. Because the effective reach of stacked convolutions is usually smaller than the theoretical one, the breakdown may already start at 2×.

Table 3: Mirror and rotation tests at 1×, $t = 1$. Model A: mean (seed 0, seed 1).

	Model A	Blank	Copy-last
$E^{(5)}$	0.0222 (0.0225, 0.0218)	0.0426	0.0641
C_M	0.00190 (0.00180, 0.00201)	0	0
E_M	0.0228 (0.0232, 0.0224)	0.0426	0.0641
C_R	0.00321 (0.00340, 0.00301)	0	0
E_R	0.0238 (0.0242, 0.0234)	0.0426	0.0641

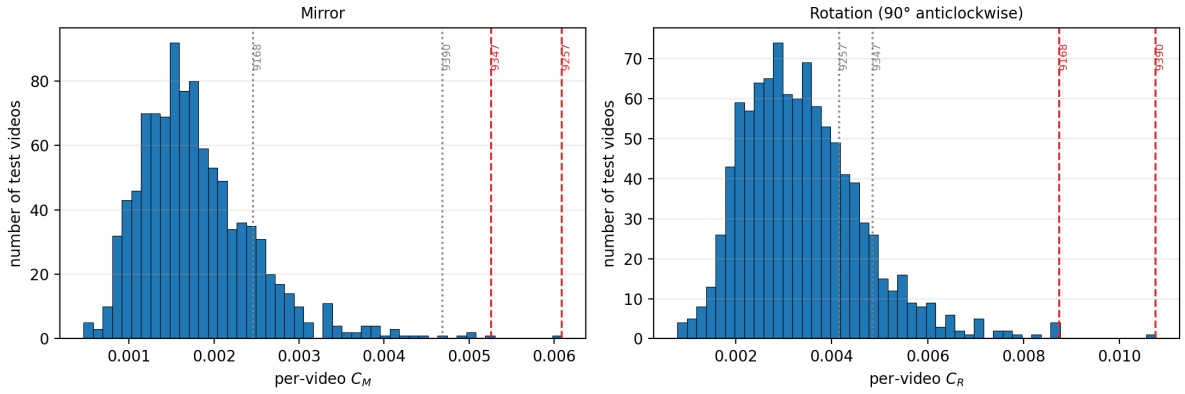


Figure 4: Per-video consistency gaps of seed-0 model A over the 1,000 test videos: C_M (left) and C_R (right). Red dashed lines mark the Figure 3 videos chosen for that transformation; grey dotted lines mark those chosen for the other one.

2.7 Q2.6

Blank and Copy-last are consistent under mirroring and rotation ($C_M = C_R = 0$), though they have much higher forecasting errors than model A (Table 3). This shows that consistency alone is not enough to judge whether a model has learned useful motion patterns.

The reason is that both baselines are consistent for trivial reasons. Blank always predicts zero-valued frames, which remain unchanged after mirroring or rotation. Copy-last repeats the final input frame, so transforming the input and then copying the last frame gives the same result as copying the last frame first and then transforming it. Neither baseline needs to learn how digits move to achieve perfect consistency. Their errors are also identical on original, mirrored and rotated clips: they are equally bad in every orientation.

Therefore, a small consistency gap does not necessarily mean that a model has learned a motion rule that generalises to transformed scenes. It may simply reflect a prediction rule that ignores motion altogether. The transformed error E_T is also needed, because it measures how closely the prediction matches the transformed ground truth. A model should ideally have both a small consistency gap C_T and a small transformed error E_T.

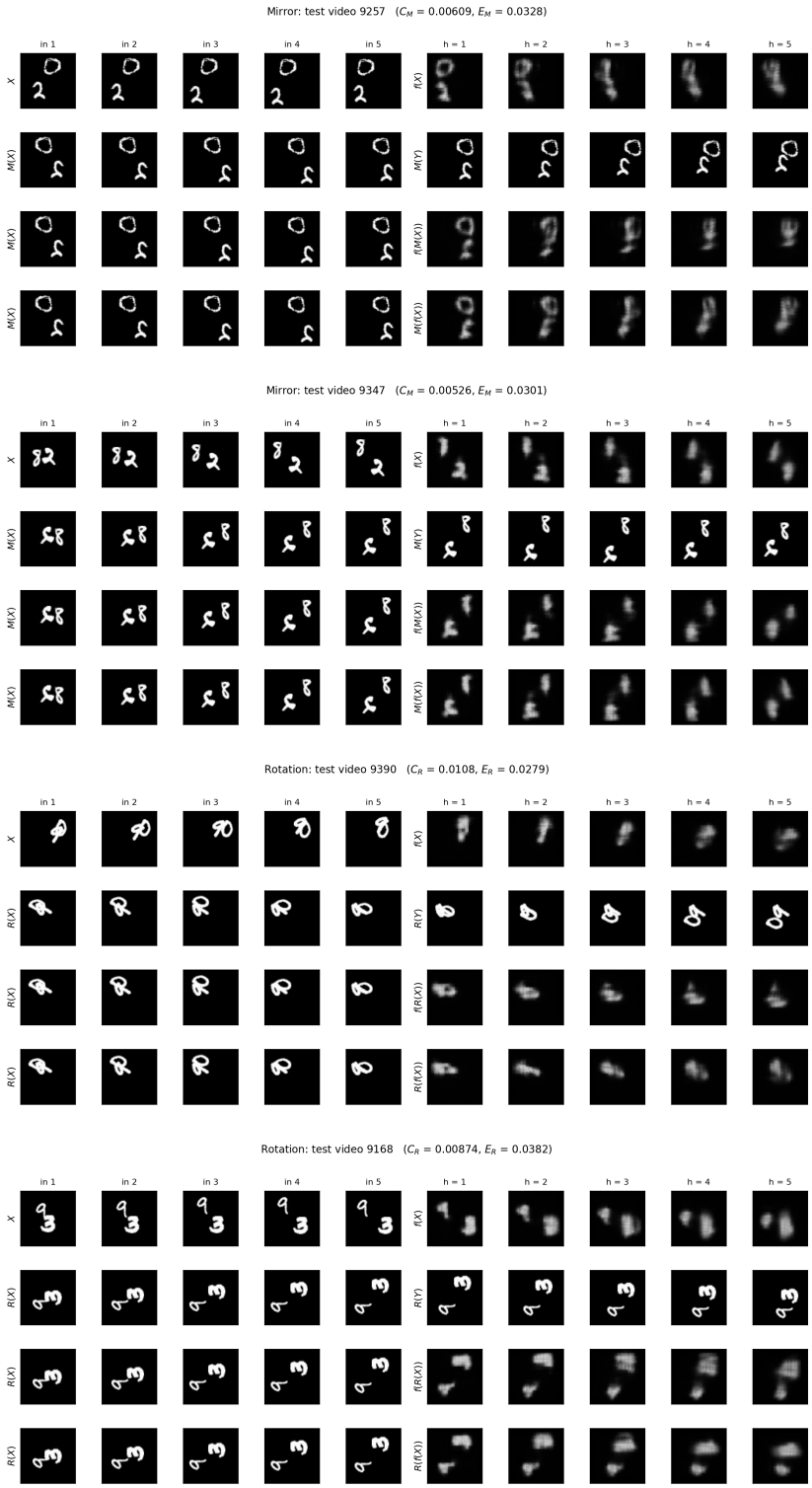


Figure 3: Seed-0 model A on the two test videos with the largest C_M (top two panels) and the two with the largest C_R (bottom two). Columns 1–5 are input frames and columns 6–10 the five forecast horizons. Rows: (1) X and $f(X)$; (2) $T(X)$ and the true future $T(Y)$; (3) $T(X)$ and $f(T(X))$; (4) $T(X)$ and $T(f(X))$. Where rows 3 and 4 disagree, the forecast depends on the digits' orientation. Animated versions: `gifs/fig3_M_video9257.gif`, `gifs/fig3_M_video9347.gif`, `gifs/fig3_R_video9390.gif`, `gifs/fig3_R_video9168.gif`.

For model A, the transformations cost little accuracy: its error rises from $E^{(5)} = 0.0222$ to $E_M = 0.0228$ under mirroring (2.8% higher) and to $E_R = 0.0238$ under rotation (7.4% higher). Its consistency gaps ($C_M = 0.00190$, $C_R = 0.00321$) are clearly above the baselines' zero, so its predictions on transformed clips do not exactly match its transformed normal predictions. On average, though, they are almost as accurate.

2.8 Q2.7

Video 9257 has the largest mirror gap of all test videos ($C_M = 0.00609$, against an average of 0.00190), and its error on the mirrored video ($E_M = 0.0328$) is well above the average (0.0228). In the true mirrored future (row 2) the two digits stay separate: the 0 drifts to the right along the top while the mirrored 2 moves up towards it and ends just below it.

Neither of the model's two accounts follows this. In both rows 3 and 4 the digits stay near the centre instead of moving right, and from the second horizon they merge into one vertical blob, even though the true digits never overlap. Beyond that, the two accounts differ in what happens to the lower digit. When the model predicts directly from the mirrored video (row 3), the mirrored 2 gets lost: the lower part of the blob fades, and the total brightness of the forecast drops from 1.16 to 0.90 times that of the true future by the fourth horizon. When the model predicts from the original video and the result is mirrored afterwards (row 4), it keeps both digits as one elongated smear stretching down and to the left, and the brightness stays between 1.14 and 1.26.

So on the mirrored video the model keeps the 0, which looks almost the same when mirrored, and loses the mirrored 2, which is not a real digit. This fits the appearance explanation in Q2.8. The disagreement also grows with every horizon, from 2.54×10^{-3} at the first horizon to 8.97×10^{-3} at the fifth, because each prediction is fed back as the next input and small early differences build on each other.

2.9 Q2.8

Rotation is slightly harder than mirroring for model A. The transformed error is 0.0238 under rotation against 0.0228 under mirroring, and both seeds agree (0.0242 vs 0.0232, and 0.0234 vs 0.0224). The difference is consistent across test videos as i averaged over both seeds, $E_R - E_M = 1.00 \times 10^{-3}$, with a 95% bootstrap interval over test videos of 8.94×10^{-4} to 1.10×10^{-3}. The consistency gap is 1.7 times larger under rotation (0.00321 vs 0.00190), and it is larger than the mirror gap in 94% of test videos. Figure 4 shows the same thing: the whole distribution of per-video gaps shifts to the right, so the extra difficulty affects almost every video rather than a few.

The difference cannot come from motion. Q2.1 showed that mirroring and rotation shift the direction statistics by almost the same amount (TV 0.071 and 0.079), because both swap the same two diagonals, and neither changes the speed. A motion effect of the same size cannot explain a gap that is 1.7 times larger. The difference must therefore come from appearance. Rotation turns vertical strokes into horizontal ones and produces shapes unlike any digit, while mirroring leaves symmetric digits such as 0, 1 and 8 almost unchanged.

The overall effect is smaller than I predicted in Q2.5 (7.4% above $E^{(5)}$, not 15–30%). A likely reason is that model A loses the digits' shape within a few frames anyway

(Figure 2). After that, its forecast is a blob whose quality depends mostly on position, which is equally hard in any orientation. In addition, model A sees digits as short stroke pieces inside 4×4 patches, and normal digits already contain strokes in many orientations, so rotated strokes are less unfamiliar than whole rotated digits.

Table 4: Speed test at $t = 1$. Model A: mean (seed 0, seed 1). Blank is included for reference.

	Model A	Copy-last	Blank
$E^{(2)}$ at 1×	0.0188 (0.0189, 0.0186)	0.0548	0.0426
$E^{(2)}$ at 2×	0.0453 (0.0452, 0.0455)	0.0661	0.0425
$E^{(2)}$ at 3×	0.0521 (0.0517, 0.0526)	0.0703	0.0426
$E^{(5)}$ at 1×	0.0222 (0.0225, 0.0218)	0.0641	0.0426
$E^{(5)}$ at 2×	0.0504 (0.0503, 0.0506)	0.0719	0.0425
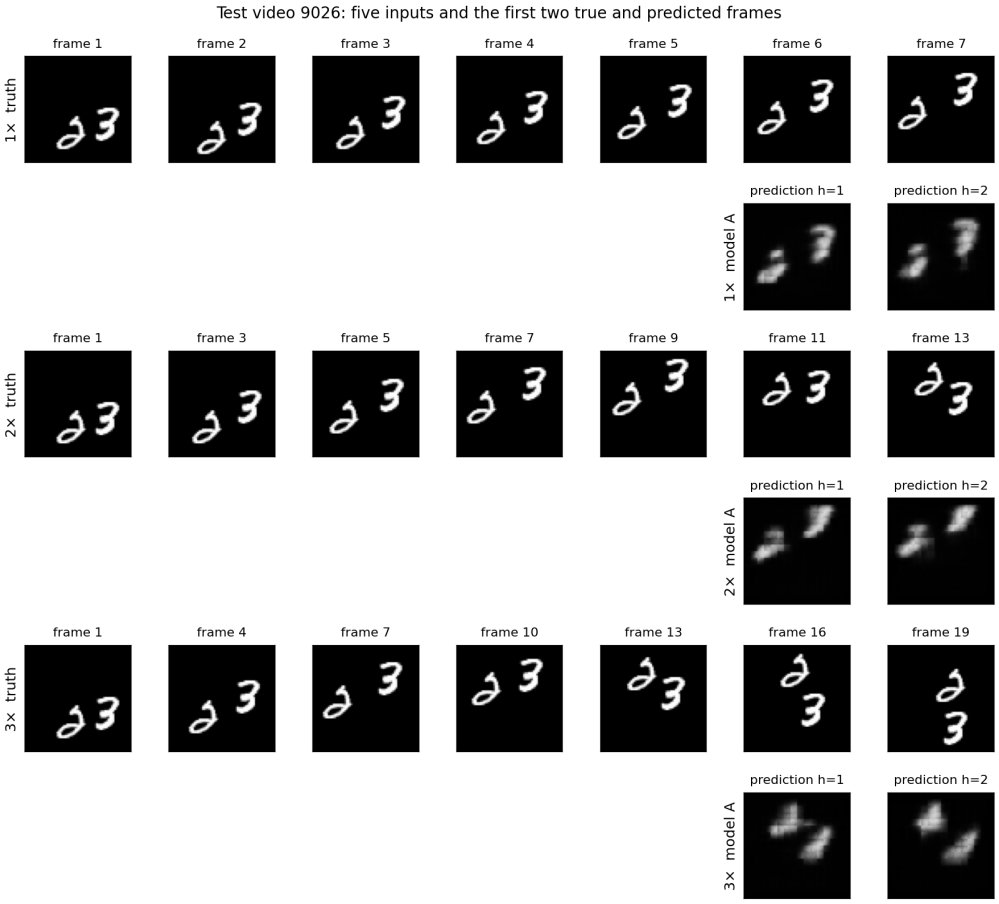


Figure 5: Seed-0 model A on test video 9026 at 1×, 2× and 3× ($t = 1$): the five input frames and the first two true frames (upper row of each pair) and model A's first two predictions (lower row). The digits do not overlap in any displayed frame. At 2× and 3× both digits bounce off a wall between the frames the model sees.

2.10 Q2.9

Model A's predictions fall apart at 2×, earlier than I predicted. Its short-term error $E^{(2)}$ rises from 0.0188 at 1× to 0.0453 at 2×, already slightly worse than predicting a blank frame (0.0425), and to 0.0521 at 3× (Table 4). About 80% of the total increase happens by 2×. The failure is already there in the first prediction (0.0410 at 2× against 0.0170 at 1×), before any of the model's own outputs are fed back, so the problem is tracking the motion within a single step, not errors building up over the rollout.

This does not agree with my reach estimate. At 2× the typical jumps between frames are $(6, 4)$, $(6, 2)$ and $(2, 8)$ px, all within the 8 px per axis estimated in Q2.3, so by that estimate model A should have coped. What the estimate missed is that reach only says what the network could connect, not what it has learned to connect. Trained only at 1×, model A has never had to use displacements larger than about 4 px. Section 4 confirms this: the same architecture trained with speed augmentation (model C) handles 2× and 3× much better (Q2.13).

Figure 5 shows test video 9026. At 1× model A moves both digits up and to the left by about the right amount, although they lose their shape and become blurred patches. At 2× and 3× the digits are heading towards the top and left walls and bounce between the frames the model sees, so in the true frames they come back towards the centre. Model A's predictions instead stay close to where the digits were last seen, as blurred, blocky patches: the digits stall. They are not duplicated, and they do not vanish.

To check that this is typical, I matched each digit in the last input frame against the true next frame and against model A's first prediction, for the digits in the 106 test videos without overlap (190–204 digits per speed). At 1× model A moves the digits 3.2 px against a true 3.6 px (median ratio 0.95). At 2× the true jump doubles to 7.1 px, but model A moves them only 2.0 px (ratio 0.28), and at 3× only 1.4 px against 10.6 px (ratio 0.15). The faster the digits really move, the less model A moves them.

So model A does not keep moving the digits at its training speed: it stalls. This is what one would expect if it can no longer connect a digit's old position, held in its state, with its new position in the input. Without that link it cannot tell which way the digit is going, and the cheapest guess under squared error is a blurred digit roughly where it was last seen. The other failure types would mean different things. A duplicated digit would mean the model treats the old and new positions as two separate digits. A vanishing digit would mean it has lost track completely and draws nothing. Neither happens here: the model still knows where the digits were, but not where they are going. From these results alone, this could be a reach problem or a lack of training on large displacements (Q2.10); Section 4 shows it is mainly the latter.

2.11 Q2.10

No. Model A was trained only at 1×, and the faster videos are both unfamiliar and further than its reach can comfortably cover, so both explanations predict the same failure. The two can be separated by changing one thing at a time. First, train the same architecture with speed augmentation, drawing the speed k uniformly from $\{1, 2, 3\}$ (model C). If C handles 2× and 3× well, the failure was unfamiliar motion: the network could track fast digits but had never been taught to. If C still fails, keep its training data and enlarge only the reach, for example with dilated convolutions (model D). If D then succeeds where

C fails, the failure was architectural: no amount of training data lets information travel further than the network can carry it in one step.

2.12 Q2.11

Model B should remove most of the mirror and rotation gap, bringing E_M and E_R close to its own $E^{(5)}$, perhaps at a small cost at 1×, but it will not help at 2× or 3×. Model C should improve 2× only partly and still stall at 3×, because more training data cannot carry information further than its reach allows. Model D, with a larger reach and the same speed augmentation as C, should fix most of the speed failure, including 3×, at little or no cost at 1×.

Table 5: Models A–D on the test set. Each entry: mean (seed 0, seed 1).

	Model A	Model B	Model C	Model D
$E^{(5)}$ at 1×	0.0222 (0.0225, 0.0218)	0.0227 (0.0232, 0.0223)	0.0270 (0.0272, 0.0268)	0.0273 (0.0277, 0.0270)
C_M	0.00190 (0.00180, 0.00201)	0.00141 (0.00122, 0.00160)	0.00111 (0.00101, 0.00122)	7.81e-04 (6.33e-04, 9.29e-04)
E_M	0.0228 (0.0232, 0.0224)	0.0228 (0.0233, 0.0223)	0.0272 (0.0274, 0.0270)	0.0275 (0.0278, 0.0272)
C_R	0.00321 (0.00340, 0.00301)	0.00147 (0.00138, 0.00156)	0.00244 (0.00227, 0.00260)	0.00181 (0.00163, 0.00199)
E_R	0.0238 (0.0242, 0.0234)	0.0227 (0.0232, 0.0222)	0.0282 (0.0283, 0.0280)	0.0284 (0.0286, 0.0282)
$E^{(2)}$ at 1×	0.0188 (0.0189, 0.0186)	0.0191 (0.0194, 0.0188)	0.0234 (0.0237, 0.0230)	0.0246 (0.0248, 0.0243)
$E^{(2)}$ at 2×	0.0453 (0.0452, 0.0455)	0.0447 (0.0459, 0.0434)	0.0289 (0.0290, 0.0287)	0.0290 (0.0292, 0.0288)
$E^{(2)}$ at 3×	0.0521 (0.0517, 0.0526)	0.0513 (0.0527, 0.0498)	0.0334 (0.0335, 0.0332)	0.0325 (0.0325, 0.0325)
Training time per epoch (s)	1.55 (1.61, 1.48)	1.79 (1.80, 1.78)	1.50 (1.50, 1.50)	1.50 (1.50, 1.50)
Inference time (s)	0.0575 (0.0575, 0.0575)	0.0567 (0.0574, 0.0559)	0.0558 (0.0558, 0.0559)	0.0554 (0.0554, 0.0554)
Parameters	480,784	480,784	480,784	480,784

Table S1: Supplementary: model D candidates on the validation split (seed 0, speed augmentation, same protocol as C), with model C for reference. Selection rule, fixed in advance: lowest validation $E^{(2)}$ averaged over 1×, 2× and 3×.

	Reach (px/axis)	Params	$E^{(2)}$ 1×	$E^{(2)}$ 2×	$E^{(2)}$ 3×	Score
C (reference)	8	480,784	0.0235	0.0290	0.0329	0.0285
Patch 4, dilations (1, 2) (chosen)	12	480,784	0.0246	0.0290	0.0319	0.0285
Patch 4, dilations (1, 3)	16	480,784	0.0279	0.0312	0.0340	0.0310
Patch 8, dilations (1, 1)	16	594,496	0.0395	0.0395	0.0395	0.0395
Patch 8, dilations (1, 2)	24	594,496	0.0377	0.0376	0.0377	0.0377

2.13 Q2.12

Model D is model A with one change: the second ConvLSTM layer uses a 3×3 kernel with dilation 2 (and padding 2) instead of dilation 1. A dilated kernel has the same nine weights but samples cells two apart, so D has exactly the same 480,784 parameters as models A–C, and C and D differ only in reach.

Using the conventions of Q2.3, one cell is still 4 px. Layer 1's convolution moves information at most 1 cell and layer 2's at most 2 cells. The new input and layer 1's previous hidden state both pass through both convolutions, so they can move $1 + 2 = 3$ cells. Layer 2's previous hidden state only passes through layer 2's convolution, so it moves 2 cells. The cell states and the 1×1 head move nothing. The longest route is therefore 3 cells, or 12 px along a row or column ($3\sqrt{2}$ cells ≈ 17.0 px diagonally), compared with

8 px (11.3 px) for model A. Combining a 1-cell and a 2-cell step reaches every offset from 0 to 3 cells, so there are no gaps.

I chose this design on the validation split from four candidates, using a rule fixed in advance: lowest validation $E^{(2)}$ averaged over the three speeds (Table S1). Dilating the second layer by 3 instead (16 px reach) was worse at every speed. The two designs with 8×8 patches (16 and 24 px) failed to train: their validation error stayed at about the level of predicting a blank frame, so they say nothing about reach. Even the chosen design only tied with model C on validation.

2.14 Q2.13

Mirror and rotation: appearance. Symmetry augmentation removes the rotation penalty. Model B's error on rotated videos ($E_R = 0.0227$) equals its error on normal ones ($E^{(5)} = 0.0227$), whereas model A's was 7.4% higher, and B's rotation gap is less than half of A's (1.47×10^{-3} against 3.21×10^{-3}). Since Q2.8 traced the rotation penalty to appearance, this is an appearance failure, and data fixes it. Symmetry augmentation also evens out the small difference in motion directions found in Q2.1, so a small motion contribution cannot be ruled out. Models C and D also have smaller gaps than A, but mainly because their predictions are blurrier, and blurrier predictions differ less from each other (Q2.6): their rotated error is still about 4% above their normal error, and only B removes the penalty.

Speed: mostly unfamiliar motion. Speed augmentation alone (A against C) cuts the short-term error by about a third at both 2× (0.0453 to 0.0289) and 3× (0.0521 to 0.0334), so most of model A's speed failure was unfamiliar motion: it had never been trained to use large displacements. Adding reach (C against D) improves 3× only slightly (0.0334 to 0.0325, consistent across seeds) and leaves 2× unchanged, so architectural reach accounts for at most a small part of the remaining failure at 3×.

What a reach failure would have looked like. Had C still failed at 3× while D succeeded, the cause would have been reach. In that case more training data could not help: the prediction at a location is computed only from inputs within reach of it, so anything further away is simply not part of the calculation. Its gradient is exactly zero, and no amount of data or training can make the network use it. That is not what happened here. Model C coped at 3× with only 8 px of reach, although the digits jump about 10 px. A likely reason is that digits are not points: a digit is about 20 px wide, so its old and new positions still overlap, and parts of the same strokes stay within reach even when its centre moves further. The remaining gap between C at 3× (0.0334) and at 1× (0.0234) may simply reflect that faster motion is harder to predict, for example because digits more often bounce between the frames the model sees.

2.15 Q2.14

Symmetry augmentation was almost free: model B's error on the original clips rose only from 0.0222 to 0.0227, while it removed the rotation penalty. Speed augmentation had a real cost: models C and D are about 22% worse than A at 1× (0.0270 and 0.0273 against 0.0222). A likely reason is that only a third of their training windows are at normal speed, and the same capacity and 30-epoch budget now has to cover three speeds; model

A itself had not finished improving at epoch 30. For C, the cost is worth it if the model has to handle different speeds: its average short-term error over the three speeds falls from 0.0387 to 0.0286, and at 2× and 3× it goes from worse than a blank frame to clearly better. If only normal-speed clips matter, model A is better. The extra reach of model D was not worth it: it gives a 3% improvement at 3× but is slightly worse than C at 1×. Inference times are almost the same for all four models; symmetry augmentation makes training about 15% slower per epoch (1.79 against 1.55 s), because every training window has to be transformed.

2.16 The final model

Why the symmetrised forecaster is exactly consistent. Take any symmetry $h \in G$. By definition,

$$\bar{f}(hX) = \tfrac{1}{8} \sum_{g \in G} g^{-1}\big(f(g\,h\,X)\big).$$

Write $g' = gh$. Because G is closed under composition and contains every inverse, the map $g \mapsto gh$ only reorders G: as g runs over all eight symmetries, so does g'. Also $g^{-1} = (g'h^{-1})^{-1} = h\,g'^{-1}$. Therefore

$$\bar{f}(hX) = \tfrac{1}{8} \sum_{g' \in G} h\big(g'^{-1}(f(g'X))\big) = h\Big(\tfrac{1}{8} \sum_{g' \in G} g'^{-1}(f(g'X))\Big) = h\big(\bar{f}(X)\big).$$

Moving h outside the sum is allowed because h only rearranges pixels, so it does not matter whether images are added before or after it is applied. With $h = M$ and $h = R$ this gives $\bar{f}(T(X)) = T(\bar{f}(X))$, so $C_M = C_R = 0$ for any f: nothing about f was used. This is also why all eight symmetries are needed. Averaging over only the original and the mirrored version would give exact mirror consistency, but not rotation consistency, because that smaller set is not closed under rotation. In practice the eight terms are added in a different order on the two sides, so rounding leaves a tiny nonzero gap (Table 6).

Choice of final model. I wrapped D+B rather than D. I made this choice before seeing Table 6, because Table 5 showed that symmetry augmentation costs almost nothing at 1× and removes the rotation penalty, so the eight orientations the wrapper averages should all be equally good. In the event, wrapping removed the difference: symmetrised D and symmetrised D+B are equal within seed variation ($E^{(5)}$ 0.0273 and 0.0274). I kept D+B, because changing the choice after seeing test results would mean selecting on the test set.

Table 6: Symmetrised models and the final model on the test set. Each entry: mean (seed 0, seed 1). Training time is that of the underlying learned model; inference time includes all eight transformations, forward passes, inverse transformations and the averaging. D+B without the wrapper is shown for reference.

	Sym. B	Sym. D	Final: sym. D+B	D+B (unwrapped)
$E^{(5)}$ at 1×	0.0221 (0.0227, 0.0216)	0.0273 (0.0276, 0.0270)	0.0274 (0.0275, 0.0274)	0.0277 (0.0277, 0.0277)
C_M	4.44e-17 (4.78e-17, 4.10e-17)	3.20e-17 (3.11e-17, 3.28e-17)	3.69e-17 (3.38e-17, 3.99e-17)	5.45e-04 (5.99e-04, 4.91e-04)
E_M	0.0221 (0.0227, 0.0216)	0.0273 (0.0276, 0.0270)	0.0274 (0.0275, 0.0274)	0.0277 (0.0277, 0.0277)
C_R	4.55e-17 (4.89e-17, 4.21e-17)	3.30e-17 (3.21e-17, 3.39e-17)	3.77e-17 (3.46e-17, 4.08e-17)	5.79e-04 (6.05e-04, 5.53e-04)
E_R	0.0221 (0.0227, 0.0216)	0.0273 (0.0276, 0.0270)	0.0274 (0.0275, 0.0274)	0.0277 (0.0277, 0.0276)
$E^{(2)}$ at 1×	0.0186 (0.0189, 0.0184)	0.0246 (0.0248, 0.0243)	0.0246 (0.0247, 0.0245)	0.0247 (0.0248, 0.0246)
$E^{(2)}$ at 2×	0.0439 (0.0452, 0.0427)	0.0290 (0.0292, 0.0288)	0.0292 (0.0295, 0.0290)	0.0294 (0.0297, 0.0292)
$E^{(2)}$ at 3×	0.0504 (0.0519, 0.0488)	0.0324 (0.0324, 0.0324)	0.0327 (0.0326, 0.0328)	0.0329 (0.0328, 0.0330)
Training time per epoch (s)	1.79 (1.80, 1.78)	1.50 (1.50, 1.50)	1.78 (1.79, 1.78)	1.78 (1.79, 1.78)
Inference time (s)	0.243 (0.243, 0.242)	0.242 (0.242, 0.242)	0.242 (0.242, 0.242)	0.0553 (0.0553, 0.0554)
Parameters	480,784	480,784	480,784	480,784



(b) Figure 5 repeated on test video 9026 at 1×, 2× and 3×.

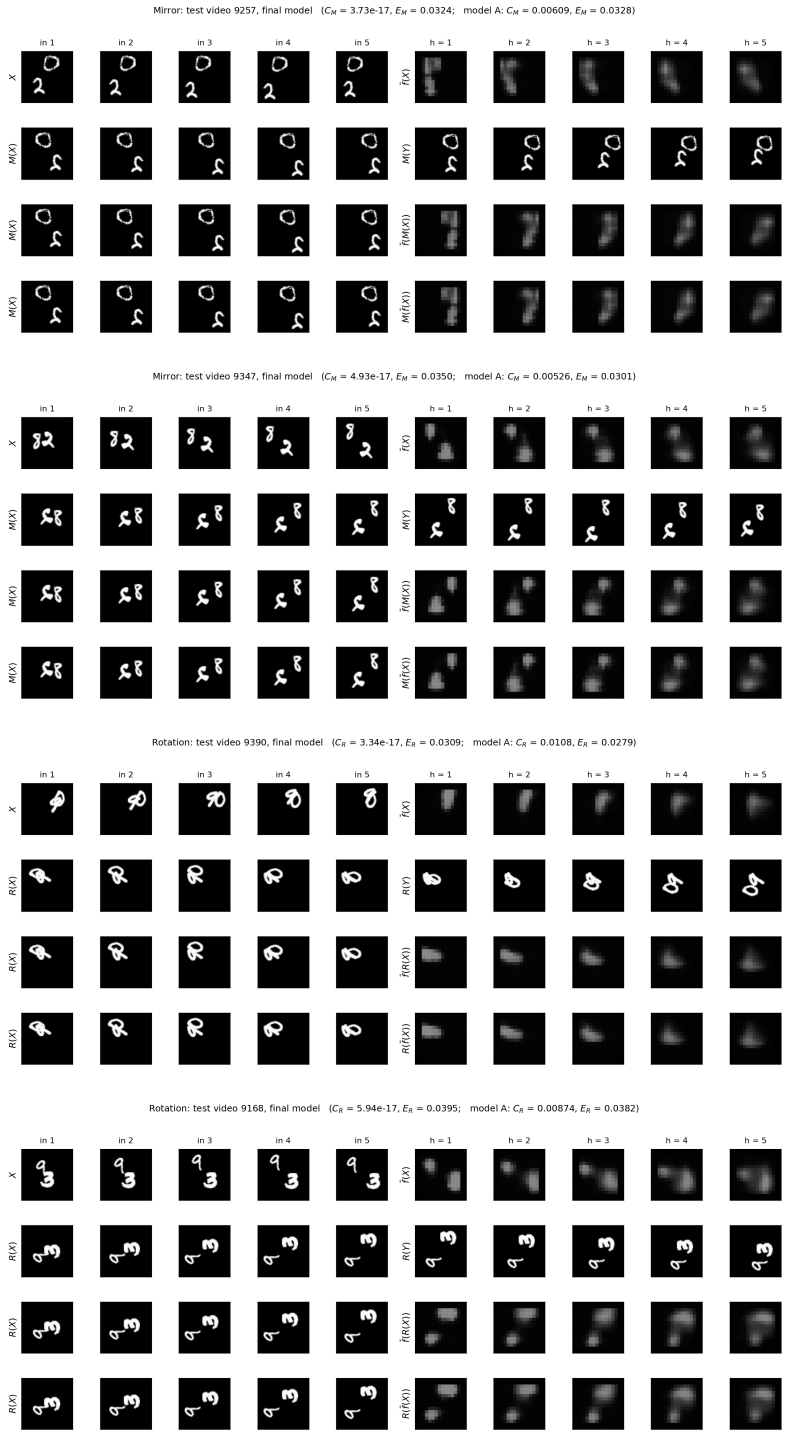
Figure 6: (continued)

Figure 6(b) repeats Figure 5 with the final model. Its forecasts are blurrier and blockier than model A's, and by the second prediction the two digits largely merge, so for this single video the improvement is hard to judge by eye. The digit-jump measurement over 190–204 digits (Q2.17) is the better evidence that the final model now follows fast motion.

2.17 Q2.15

Consistency. Built-in symmetry is exact; learned symmetry is not. Model B's gaps are 1.41×10^{-3} (mirror) and 1.47×10^{-3} (rotation), smaller than model A's but clearly not zero, and even D+B, trained with both augmentations, keeps gaps of about 5.5×10^{-4}. Every symmetrised model has gaps of about 4×10^{-17}, which is rounding error. On an untrained network the wrapper's largest gap was 5.96×10^{-8}, one rounding step of 32-bit arithmetic, against about 4.5×10^{-2} without it. Built-in symmetry is exact because it follows from how the average is built (see the argument above), whatever the model has learned. Augmentation only shows the model transformed videos and relies on it generalising.

Accuracy. Because a symmetrised model is exactly consistent, and mirror and rotation only rearrange pixels, its error on transformed clips must equal its error on the originals: $E_M = E_R = E^{(5)}$ by construction. Learned symmetry comes close (model B: $E_M =$



(a) Figure 3 repeated on the same four test videos. Rows 3 and 4 are now identical up to rounding. Animated versions: `gifs/fig6a_M_video9257.gif`, `gifs/fig6a_M_video9347.gif`, `gifs/fig6a_R_video9390.gif`, `gifs/fig6a_R_video9168.gif`.

Figure 6: The final model (symmetrised D+B, seed 0) on the videos of Figures 3 and 5.

0.0228, $E_R = 0.0227$, $E^{(5)} = 0.0227$). Wrapping did not cost accuracy on the original clips; it improved it slightly (0.0221 against 0.0227 for B, and 0.0274 against 0.0277 for D+B).

Cost. The wrapper needs no training, but eight forward passes. Predicting the test set took 0.242 s against 0.055 s, 4.4 times slower. It is not 8 times slower because the eight orientations are processed together as one batch on the GPU. Symmetry augmentation costs nothing at inference but slows training a little (1.79 against 1.55 s per epoch).

When built-in symmetry hurts. Averaging blurs the forecast, but squared error rewards hedging: the error of an average of eight forecasts is never larger than the average of their eight errors. Wrapping can therefore only lose to the plain model when the model is much worse in some orientations than in the original one, so that one good forecast is averaged with several poor ones, or when the world itself is not symmetric, for example under gravity or with written text, so that the wrapper imposes a symmetry the data does not have. Neither happened here. Even model D, never trained on transformed videos and 4% worse on rotated ones, lost nothing when wrapped ($E^{(5)} = 0.0273$ in both cases). The final model's forecasts in Figure 6 do look softer and blockier than model A's in Figure 3, but part of that comes from speed augmentation (Q2.14), and squared error does not penalise it.

2.18 Q2.16

Partly. On video 9257 the final model's two accounts now agree exactly ($C_M = 3.73 \times 10^{-17}$, against 6.09×10^{-3} for model A), and the mirrored 2 no longer fades: the forecast on the mirrored video keeps between 1.08 and 1.29 times the true future's brightness, where model A's fell to 0.90. The motion itself is still wrong (Figure 6(a)). At the first horizon the forecast sits roughly where the two digits are, but from the second horizon the 0 and the 2 merge into one diagonal smear, and the 2 never moves up under the 0 as it does in the true future. Its accuracy on the mirrored clip is therefore barely better than model A's ($E_M = 0.0324$ against 0.0328).

Each change fixed a different part of the failure. Symmetry augmentation fixed the fading and most of the disagreement: model B keeps the brightness above 1.05, cuts the gap to 2.28×10^{-3}, and is the most accurate model on this clip ($E_M = 0.0268$). Speed augmentation did nothing for this normal-speed clip and made the forecasts worse ($E_M = 0.0347$ for C and 0.0342 for D). Their brightness of about 1.2–1.3 suggests intensity spread over a larger area, that is, blurrier forecasts: the same 1× cost as in Q2.14. The wrapper then removed the remaining gap exactly. The final model inherits the consistency of the symmetry changes, but also the blur of the speed changes, which is why it does not beat model B here. The same holds across Figure 6(a): on the other three videos the final model is perfectly consistent but less accurate than model A (0.0350 against 0.0301 for video 9347, 0.0309 against 0.0279 for 9390, and 0.0395 against 0.0382 for 9168).

2.19 Q2.17

Partly. The final model has learned a motion rule that transfers across speeds, but its orientation robustness is built in, not learned: because the wrapper makes it exactly consistent, its error on mirrored and rotated clips equals its error on the originals by

construction, so those results prove nothing about learning. The real evidence is speed. Model A left digits almost where they were at 3× (1.4 px against a true 10.6 px); the final model moves them 7.3 px against 7.1 at 2× and 8.5 against 10.6 at 3×, and its short-term error at 2× and 3× is about 35% lower. A skeptic should be shown speeds it never trained on, such as 4× or 1.5×, and clips with one or three digits. One test it still fails: at 3× it still undershoots the motion, and its error stays a third higher than at 1×.

3 Use of AI

Tools used. Google Gemini, and chatgpt to format latex for diagram and python syntax's for coding part. For Question 2 used it as a tutor and coding assistant where i got lost while coding, in one long conversation. It explained the question and every number in this report comes from those experiments i did.